

LESSON 01

Mobile Testing: APPIUM

 *NeoTechAcademy*

Lesson Outline

01 Mobile Testing & Testing Tools

02 Appium

03 Appium Features

04 How Appium Works

05 Appium Inspector

Mobile Testing & Testing Tools

- Mobile application testing is an interaction by which application programming produced for mobile devices is tried for its usefulness, convenience, and consistency.
- Mobile applications either come pre-introduced or can be introduced from portable programming dissemination stages.
- Mobile testing can be manual or automated.
- Mobile testing tools are programmed to test portable applications.

Mobile Testing & Mobile Testing Tools

→ This classification incorporates:

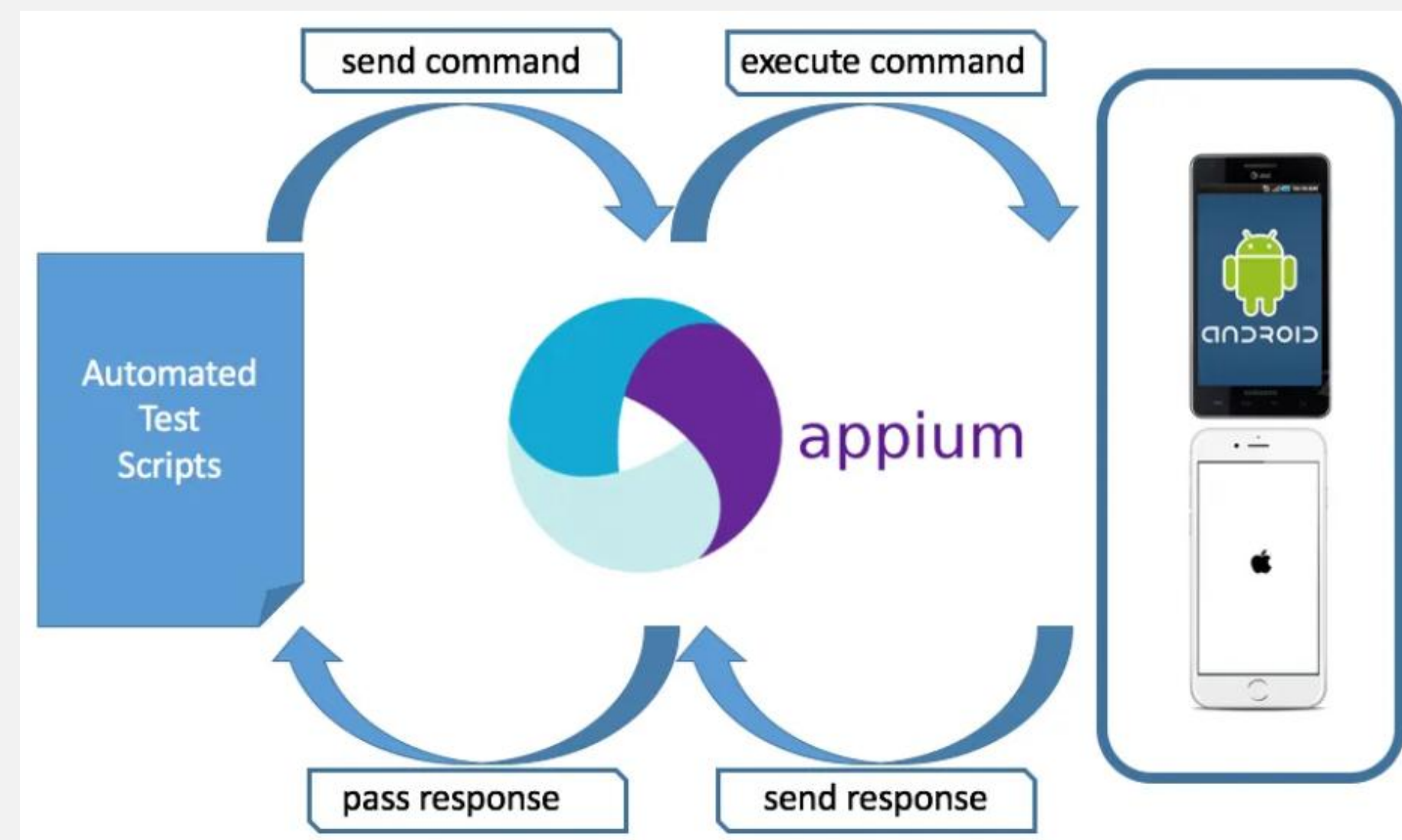
- cloud-based testing devices,
- application conveyance instruments,
- crash-revealing tools,
- execution testing instruments,
- mobile phone emulators,
- ...

Mobile Application Types

- Mobile application is a software designed to run on mobile devices.
- Primarily, there are four types of mobile applications:
 - Native apps: designed for specific mobile operating systems like Android, iOS, or Windows.
 - Web apps: optimized for mobile devices and they are accessible through a web browser.
 - Hybrid apps: combine elements of both native and web apps, using a single codebase to run on multiple platforms. These are deployed on container that uses mobile WebView object.
 - Progressive Web Apps (PWA): designed to provide a native app like experience.

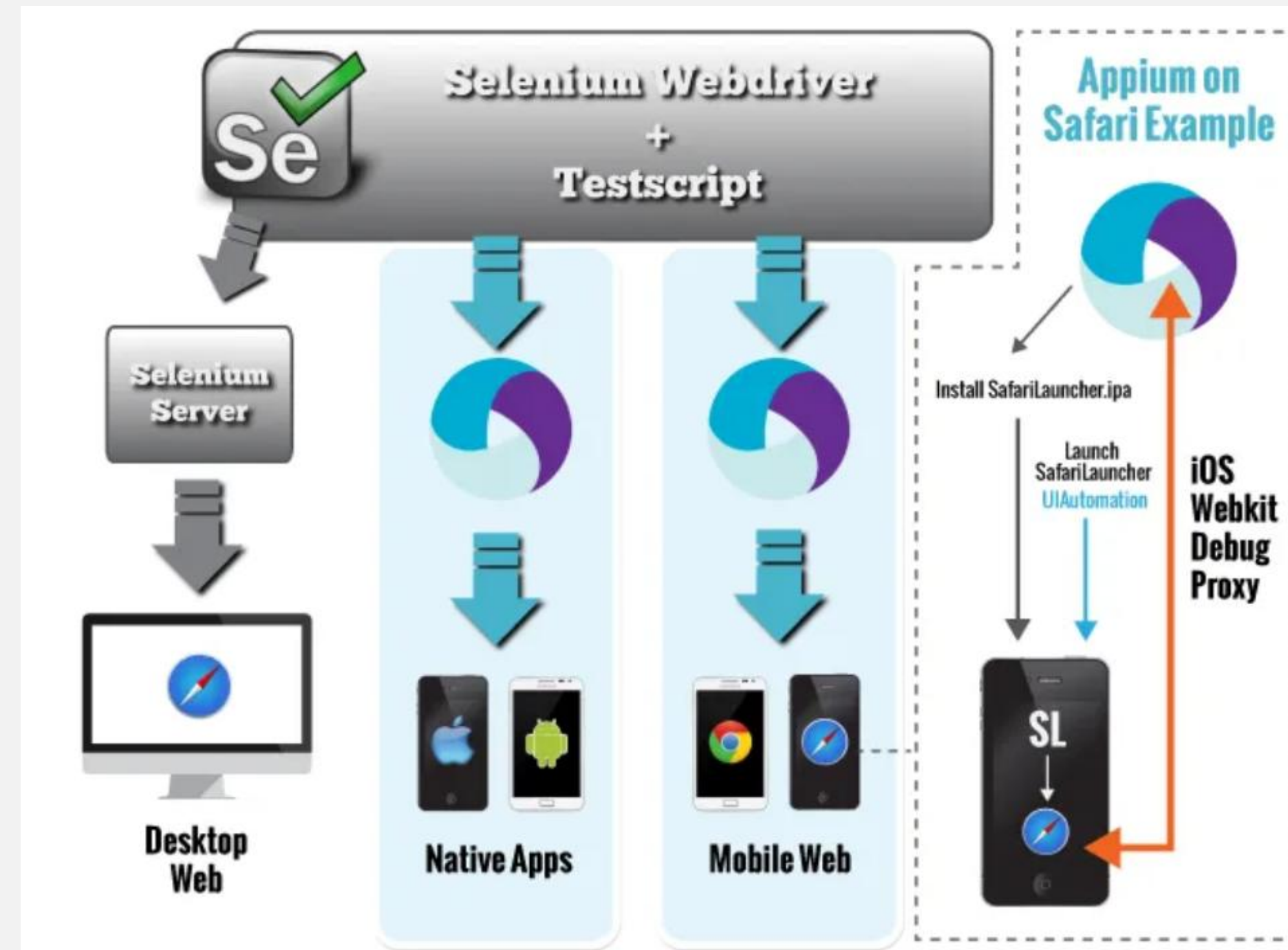
Appium

- Appium is an open-source automation framework that simplifies the process, by controlling and interacting with mobile apps on iOS, Android, etc. devices all through API.
- Appium aims to support UI automation of many different platforms (mobile, web, desktop, etc.). Not only that, but it also aims to support automation code written in different languages (JS, Java, Python, etc.)



Appium

- Appium uses the WebDriver protocol to interact with mobile apps
 - making it compatible with all programming languages supporting WebDriver
- Appium is often combined with Selenium to provide efficient and scalable cross-platform mobile automation.



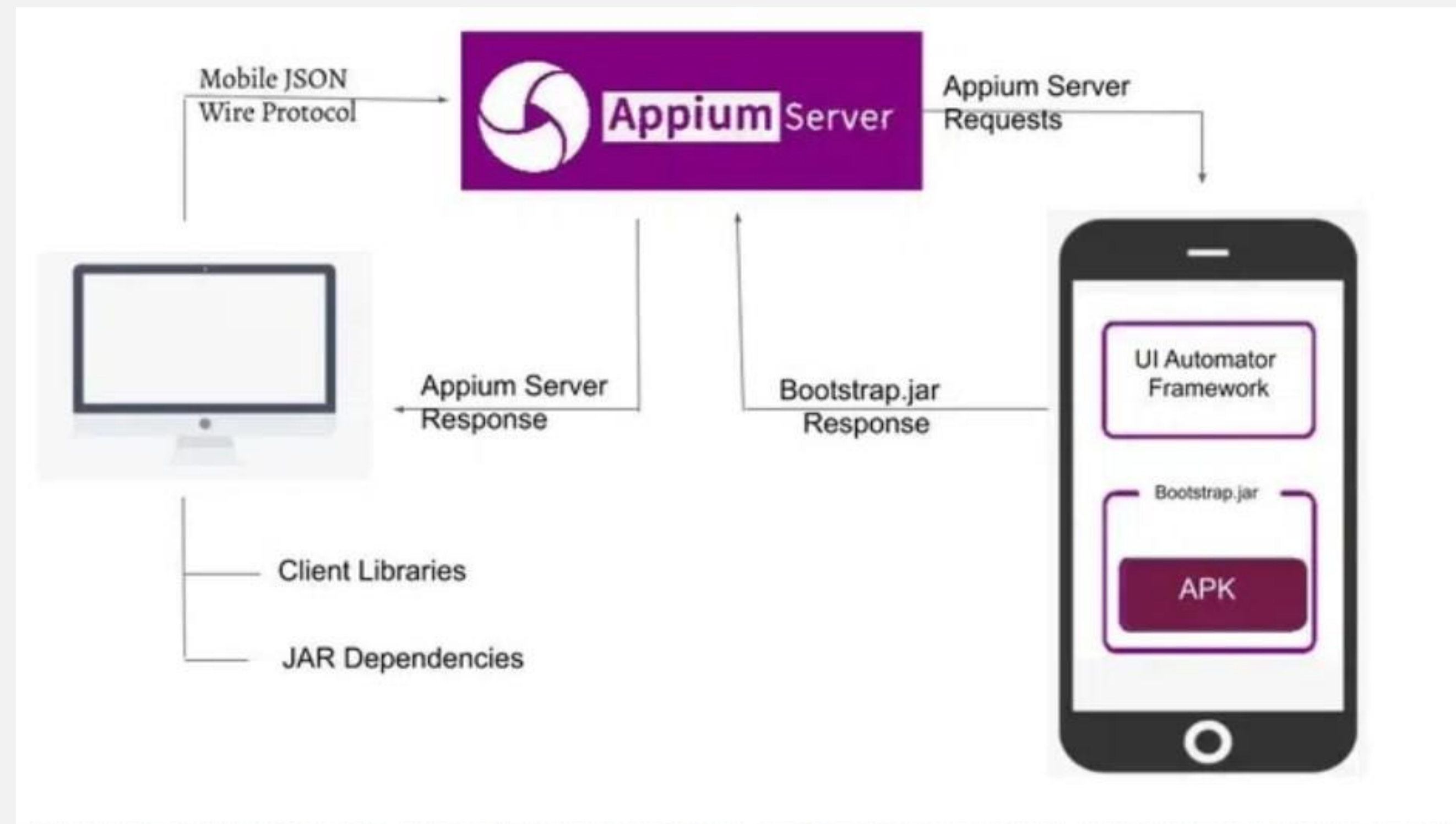
Appium Features

→ Appium Features

1. Cross-platform compatibility
2. Open source
3. Supports multiple programming language
4. No app modification required
5. Integration with CI tools
6. Supports native, hybrid and mobile Web applications
7. Real device and emulator/simulator testing
8. Built-in Inspector for element identification
9. Parallel execution of tests
10. Wide range of Appium client libraries

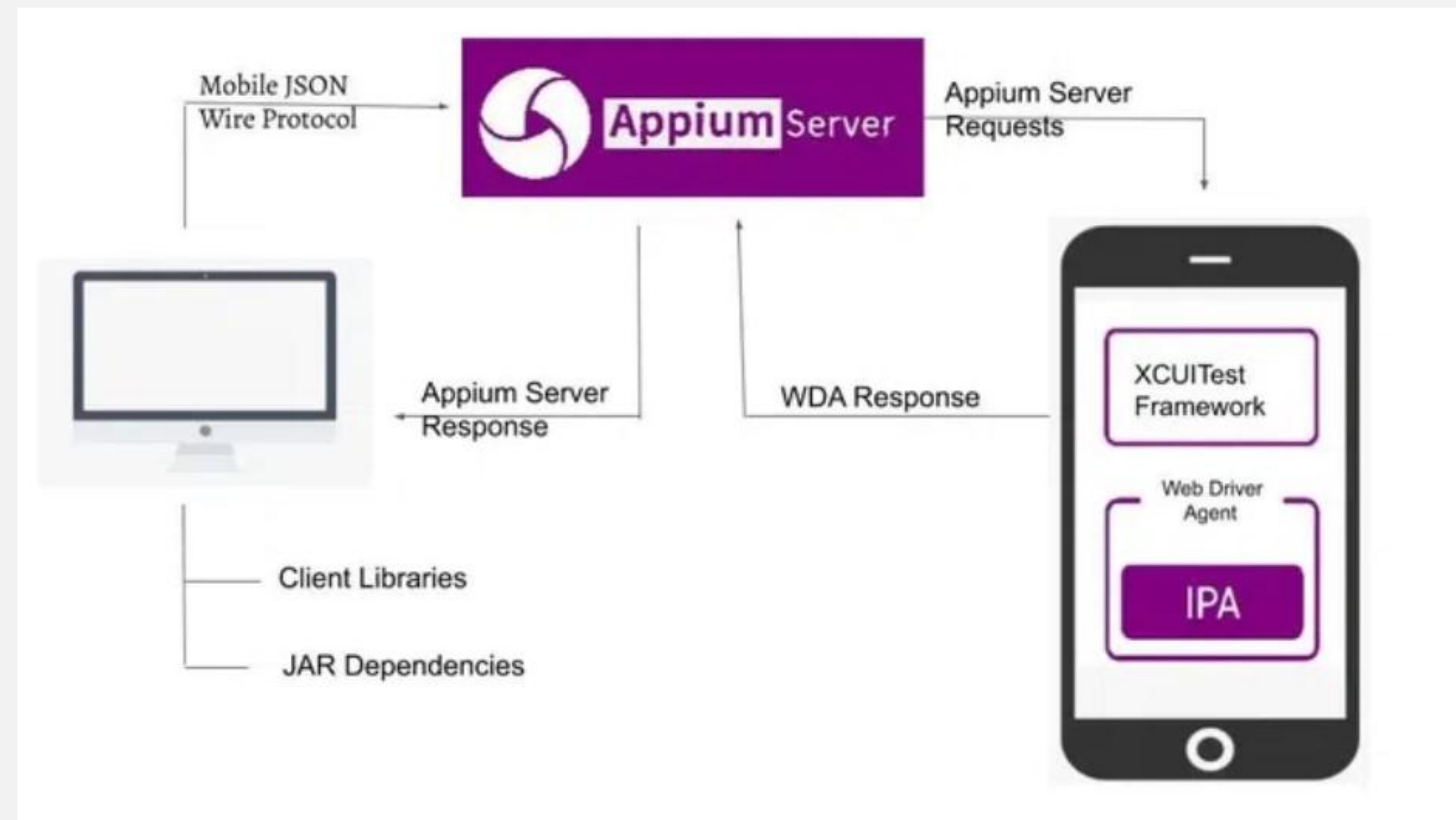
How Appium Works on Android

→ For Android, Appium uses UI Automator API's to interact with UI factors of operation Under Test.



How Appium Works on iOS

→ For iOS, Appium uses Apple's native XCUITest API to interact with UI factors of operation Under Test.



What/How to Install?

1. JDK – For Java users ✓
2. Node.js ✓
3. npm ✓
4. Appium ✓
5. Android Studio ✓
6. Appium-doctor ✓
7. Appium Inspector ✓
8. Set JAVA_HOME and ANDROID_HOME ✓
9. Configure Android Virtual Device (AVD) from Android Studio ✓
10. Check the completeness/success of your installation by running appium-doctor command ✓

NOTE: for Mac user, you may need to install carthage using homebrew. To do that run the following command:

```
brew install carthage
```

Appium Client-Server

- Appium requires both client and server agents to be installed/configured so that we can run our test cases.
- Appium Server does the communication between your codes and the platform/mobile device.

```
npm i appium -g  
or  
npm i --location=global appium
```

- Appium client for Java is a library added as a dependency to enable test cases composition.
- Example for Maven dependency

```
<dependency>  
  <groupId>io.appium</groupId>  
  <artifactId>java-client</artifactId>  
  <version>9.2.3</version>  
  <scope>test</scope>  
</dependency>
```

Appium Test Structure

→ UiAutomator2 Driver is used for Android devices to link both URL and options, and set the desired capabilities.

```
UiAutomator2Options options = new UiAutomator2Options();  
options.setUdid("MyDevice123");
```

→ The URL defines where the driver object is created, likely the local host:

```
url = new URL("http://127.0.0.1:4723/");  
It is safer to use  
url = new URL("http://localhost:4723/");
```

Appium Project: pom.xml

```
<properties>
  <maven.compiler.source>17</maven.compiler.source>
  <maven.compiler.target>17</maven.compiler.target>
</properties>

<dependencies>
  <dependency>
    <groupId>io.appium</groupId>
    <artifactId>java-client</artifactId>
    <version>9.2.3</version>
    <scope>test</scope>
  </dependency>

  <!--https://mvnrepository.com/artifact/org.junit.
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.11.0-M2</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

Appium Inspector

- Appium Inspector is a tool used in mobile application testing and development.
- It is specifically designed for automating mobile apps on various platforms such as iOS, Android, and Windows using the Appium framework.
- We can define Desired Capabilities sets according to the tested devices and cases.
- We can either define the capabilities using key-value entries or in entering them directly in the JSON Representation field.

Appium Inspector

Appium

FileViewHelp

Appium Server

Select Cloud Providers

Remote Host

127.0.0.1

Remote Port

4723

Remote Path

/

☐ SSL

>

Advanced Settings

Desired Capabilities

Saved Capability Sets 1

Attach to Session...

platformName	text	Android
appium:deviceName	text	emulator-5554
appium:platformVersion	text	15
appium:automationName	text	UiAutomator2

☒ Automatically add necessary Appium vendor prefixes on start

JSON Representation

{

"platformName": "Android",

"appium:deviceName": "emulator-5554",

"appium:platformVersion": "15",

"appium:automationName": "UiAutomator2"

}

Desired Capabilities Documentation

Save

Save As...

Start Session

Locator Strategies

- **ID: resource-id** is used as an element identifier for Android and **name** is used for iOS

```
driver.findElement(By.id("account-name"));
```

- **Class Name:** For iOS, Class Name is represented as the full name of the XCUI element and begins with XCUIElementType. Android, the Class Name is called out as the full name of the UIAutomator2 class.

```
driver.findElementsByClassName("android.widget.TextView");
```

- **Name:** Element will be identified using the name of the field.

```
driver.findElementByName(String selector);
```

- **Xpath:** Xpath in Appium analyzes the XML structure of the app and then locates the element.

```
driver.findElementByXPath("//XCUIElementTypeButton)[1]");
```

- **Accessibility ID:** For iOS the default Accessibility ID is set to the name of the UI element. For Android, the value of Accessibility is same as the value of the attribute "content-desc"

```
driver.findElement(new AppiumBy.ByAccessibilityId("plus"));
```

Locator Strategies

- **Android UI Automator:** This locator is Android-specific.

```
String selector = "new UiSelector().text(\"Cancel\")\n                .className(\"android.widget.Button\")";
```

```
MobileElement element = (MobileElement)\ndriver.findElement(MobileBy.AndroidUIAutomator(selector));
```

- **Android View Tag (Espresso Only)**

- **iOS UI Automation:**